

INGENIERÍA DE SOFTWARE

PRÁCTICA DE LABORATORIO SESIÓN 10



Proyecto: Lab10_Escena3D

Repositorio de GitHub: https://github.com/JosueRamosS/Lab10_Escena3D

Nombre del estudiante: Josué Carlos Alberto Ramos Suyoc

Curso: Desarrollo de Videojuegos

Docente: Marco Antonio Camacho Alatrística

Arequipa – Perú

2026

Descripción del trabajo realizado

A continuación describo lo que hice en cada fase de la práctica. La escena se construyó en Unity 6 con el Universal Render Pipeline (URP). En algunos pasos tuve que adaptar las instrucciones a la versión actual de Unity; señalo esos ajustes para poder justificarlos.

Fase 1: Configuración del proyecto 3D con URP

Creé el proyecto en Unity Hub usando la plantilla Universal 3D (que es la plantilla 3D (URP) en Unity 6), con el nombre Lab10_Escena3D y en una ruta sin espacios ni caracteres especiales. Luego verifiqué en Edit > Project Settings > Graphics que el proyecto realmente estuviera usando un asset de URP, para asegurarme de que el render fuera el correcto. Finalmente organicé la carpeta Assets en cinco subcarpetas (Materials, Textures, Skyboxes, Scripts y Scenes) y guardé la escena como Escena3D_Lab10.

Fase 2: Creación y transformación de objetos primitivos

Limpié la escena dejando solo la Main Camera y la Directional Light, y creé un GameObject vacío llamado Escena_Contenedor en el origen (0,0,0) para agrupar todo. Dentro de él construí la escena: un suelo (Plane) llamado Suelo_Principal, una plataforma elevada (Cube) llamada Plataforma_Central, y tres props sobre la plataforma: una caja (Prop_Caja_01), una esfera (Prop_Esfera_01) y una columna (Prop_Columna_01), cada uno con su posición, rotación y escala indicadas. Por último posicioné la Main Camera en (-4, 4, -6) con rotación (20, 25, 0) para encuadrar bien la escena.

Fase 3: Materiales con shader URP/Lit

Creé cinco materiales con el shader Universal Render Pipeline/Lit y configuré en cada uno el color base (Base Map), el Metallic y el Smoothness según los valores pedidos: un suelo gris mate, una plataforma azul grisácea, una caja marrón metálica, una columna plateada muy reflectante y una esfera cian. A la esfera además le activé la propiedad Emission con un color cian de intensidad 1.5. Al final asigné cada material a su objeto correspondiente arrastrándolo sobre él.

Fase 4: Importación y aplicación de texturas PBR

Descargué de Poly Haven una textura de suelo PBR en resolución 1K (Floor Klinkers 04), con sus mapas de albedo, normal (GL), roughness y ambient occlusion. Configuré el normal map cambiando su Texture Type a Normal map antes de usarlo, que es un paso clave para que el relieve funcione. En el material del suelo asigné el albedo al Base Map, el normal map a Normal Map y el ambient occlusion al Occlusion Map, y ajusté el Tiling a 3x3.

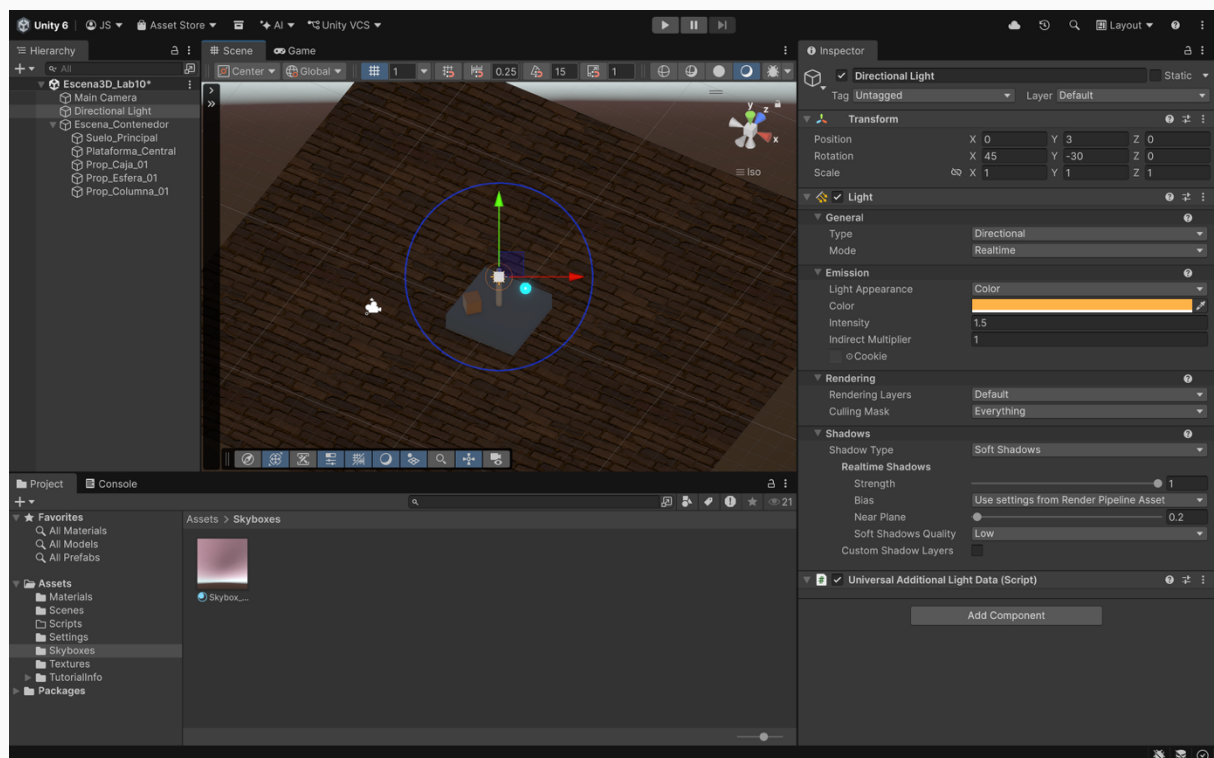
Adaptación de Unity 6: el shader URP/Lit no tiene una entrada separada para el mapa de Roughness (la nitidez se controla con el slider Smoothness). Por eso, en lugar de enchufar el mapa de roughness, bajé el Smoothness a 0 para dejar el suelo mate como concreto, logrando el mismo resultado visual.

Fase 5: Skybox procedural e iluminación ambiental

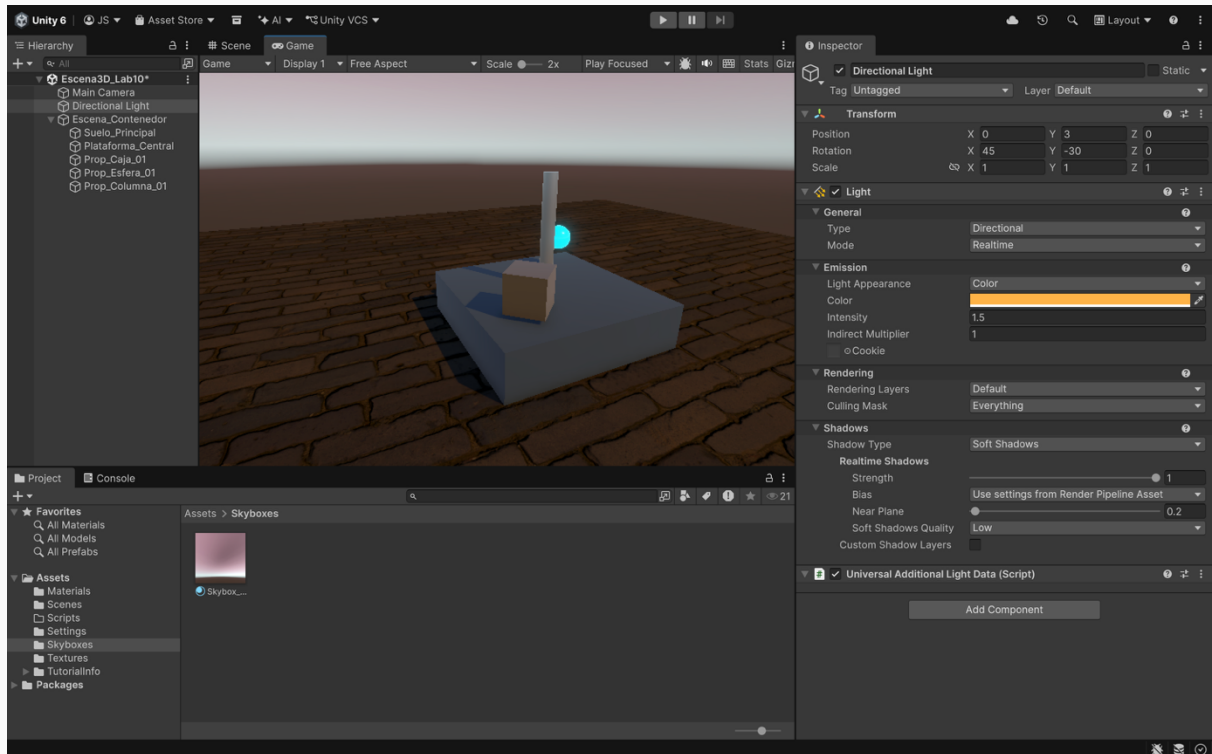
Creé un material de skybox con el shader Skybox/Procedural llamado Skybox_Atardecer, con tinte naranja cálido, y lo asigné en Window > Rendering > Lighting > Environment, verificando que el Sun Source apuntara a la Directional Light. Configuré la iluminación ambiental con Source = Skybox e intensidad 1. Para que la luz fuera coherente con el atardecer, ajusté la Directional Light a un color naranja dorado, intensidad 1.5 y una rotación de sol bajo (45, -30, 0).

Adaptación de Unity 6: la luz venía en modo Filter and Temperature, así que cambié Light Appearance a Color para poder fijar el tono exacto. La opción de Realtime Global Illumination no aparece en Unity 6 porque esa función fue retirada, por lo que ya está efectivamente desactivada, tal como pedía la práctica.

Escena 3D completa con el suelo, la plataforma, los tres props y el cielo naranja de fondo, vista desde la ventana Scene.



Vista opcional de la misma escena renderizada por la cámara del juego (evidencia adicional).



Fase 6: Cambio de skybox en tiempo de ejecución (C#)

Creé un segundo skybox nocturno (Skybox_Noche) con tono azul oscuro y exposición baja, y escribí un script en C# llamado SkyboxController. El script revisa en cada fotograma (Update) si se pulsa la tecla N y, cuando ocurre, llama a un método que alterna entre el skybox de día y el de noche, recalculando la iluminación ambiental con DynamicGI.UpdateEnvironment() solo en ese momento. Adjunté el script a un GameObject vacío llamado GameManager y asigné el skybox de atardecer al campo Skybox Dia y el de noche al campo Skybox Noche.

Adaptación de Unity 6: como el script usa la clase Input clásica (Input.GetKeyDown), configuré Active Input Handling en Both dentro de Project Settings > Player, para que esa API siguiera funcionando sin tener que reescribir el código.

Código del script SkyboxController.cs:

```
using UnityEngine;

public class SkyboxController : MonoBehaviour
{
    [Header("Materiales de Skybox")]
    public Material skyboxDia;
    public Material skyboxNoche;

    [Header("Estado inicial")]
    public bool esDeNoche = false;

    void Update()
    {
        // Presionar la tecla N para alternar entre Skybox de dia y de noche
        if (Input.GetKeyDown(KeyCode.N))
        {
            AlternarSkybox();
        }
    }

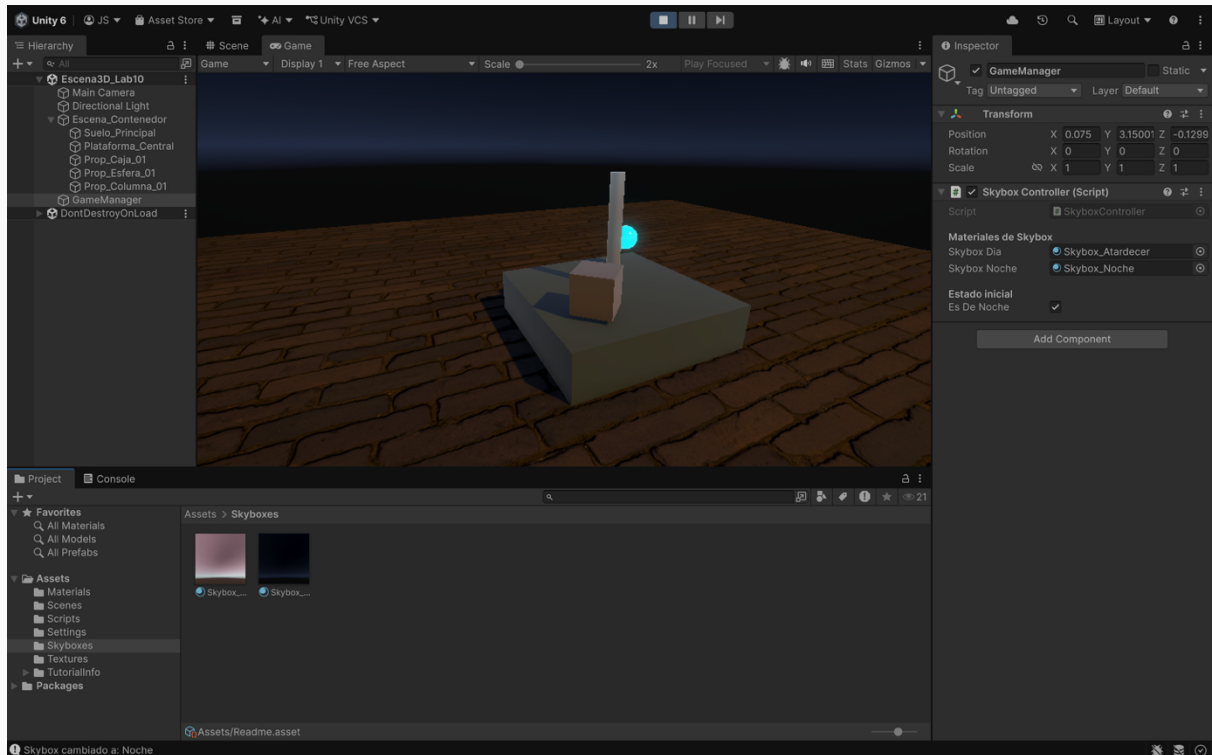
    void AlternarSkybox()
    {
        esDeNoche = !esDeNoche;

        if (esDeNoche)
        {
            RenderSettings.skybox = skyboxNoche;
        }
        else
        {
            RenderSettings.skybox = skyboxDia;
        }

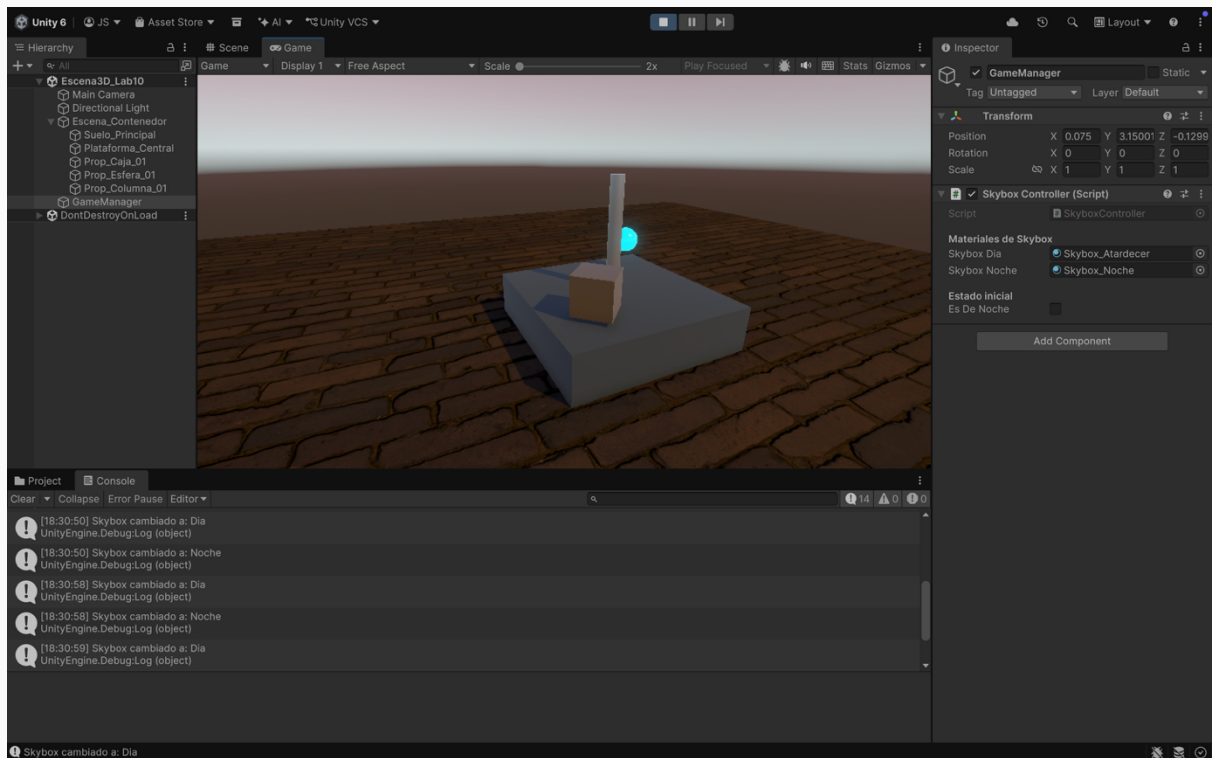
        // Recalcular la iluminacion ambiental con el nuevo Skybox
        DynamicGI.UpdateEnvironment();

        Debug.Log("Skybox cambiado a: " + (esDeNoche ? "Noche" : "Dia"));
    }
}
```

Juego en ejecución (botón Play activo) tras pulsar la tecla N, mostrando el cielo nocturno azul oscuro.



Consola mostrando al menos dos mensajes "Skybox cambiado a: ..." registrados por Debug.Log.



Preguntas de reflexión

Pregunta de reflexión 1

¿Qué ventaja ofrece agrupar todos los objetos bajo el GameObject vacío Escena_Contenedor? ¿Qué ocurriría con los objetos hijos si mueve o rota el contenedor?

Agrupar todo bajo un GameObject vacío me da dos ventajas. La primera es de organización: la jerarquía queda ordenada y puedo seleccionar, ocultar o activar toda la escena de una sola vez. La segunda es que puedo tratar todo el conjunto como una sola pieza, porque al ser hijos del contenedor, sus posiciones, rotaciones y escalas se miden respecto al contenedor (su padre) y no respecto al mundo.

Por eso, si muevo o roto el contenedor, todos los hijos se mueven y rotan junto con él, manteniendo intactas sus posiciones relativas entre sí (la caja sigue sobre la plataforma, la esfera al lado, etc.). Es como mover una bandeja con vasos encima: muevo la bandeja y todo lo de arriba viaja igual, sin tener que recolocar cada objeto uno por uno.

Pregunta de reflexión 2

La esfera tiene Emission con intensidad 1.5, pero puede que no ilumine los objetos cercanos. ¿A qué se debe? ¿Qué configuración de iluminación sería necesaria para que la emisión afecte al entorno?

Esto pasa porque la propiedad Emission solo hace que la superficie de la esfera se vea brillante por sí misma, como si estuviera encendida, pero no la convierte en una fuente de luz que reparta iluminación a su alrededor. La esfera brilla, pero la caja y la plataforma de al lado no reciben esa luz.

Para que la emisión sí ilumine a los objetos cercanos, Unity necesita tener activado y calculado el sistema de iluminación global (Global Illumination), que es el que simula cómo la luz rebota de una superficie a otra. En esta práctica la Realtime Global Illumination está desactivada (de hecho en Unity 6 fue retirada), así que la emisión se queda en simple brillo. Para lograrlo tendría que hornear (baking) la iluminación global con la esfera contribuyendo a la GI, o bien, de forma más sencilla, añadir una luz real (por ejemplo un Point Light) dentro de la esfera para que ella sí proyecte luz cian sobre el entorno.

Pregunta de reflexión 3

Compare el suelo con textura PBR completa (albedo + normal map) contra la plataforma que solo tiene color base. ¿Qué diferencias percibe? ¿En qué tipo de superficie no haría falta un Normal Map?

La plataforma, al tener solo un color base plano, se ve uniforme y artificial: la luz le pega de forma pareja, sin ningún detalle. En cambio el suelo con textura PBR se ve mucho más creíble, porque el albedo le da variación de color real (manchas, tonos, suciedad) y el normal map le agrega relieve simulado: bajo la luz aparecen pequeñas sombras y brillos sobre las irregularidades, así que el suelo parece tener rugosidad aunque la geometría siga siendo un plano liso.

Lo importante del normal map es que engaña al cálculo de luz: no añade geometría nueva, solo cambia hacia dónde mira cada punto de la superficie para que la luz rebote como si hubiera relieve. No haría falta un normal map en superficies que de verdad son lisas (un vidrio, un espejo, un metal pulido o una pared limpia), en juegos con estilo estilizado o low-poly, o

en objetos muy lejanos o pequeños en pantalla donde ese relieve no se alcanzaría a percibir y solo gastaría rendimiento.

Pregunta de reflexión 4

¿Qué impacto tendría sobre el rendimiento llamar a `DynamicGI.UpdateEnvironment()` dentro de `Update()` en cada fotograma en lugar de solo al cambiar? ¿Cómo está resuelto en el script?

`DynamicGI.UpdateEnvironment()` es una operación costosa, porque obliga a Unity a recalcular toda la iluminación ambiental que proviene del skybox. Como `Update()` se ejecuta en cada fotograma (decenas de veces por segundo), poner esa llamada directamente en `Update()` haría que Unity rehiciera ese cálculo pesado 60+ veces por segundo aunque el cielo no haya cambiado, desperdiciando recursos y bajando los FPS, con posibles tirones.

En el script está bien resuelto porque `UpdateEnvironment()` no está dentro de `Update()`, sino dentro del método `AlternarSkybox()`. `Update()` solo se dedica a vigilar si se pulsó la tecla N, y únicamente cuando eso ocurre llama a `AlternarSkybox()`. Así, el recálculo pesado se ejecuta solo en el momento exacto del cambio de cielo y no en cada fotograma: se separa detectar el evento (barato) de reaccionar al evento (caro).